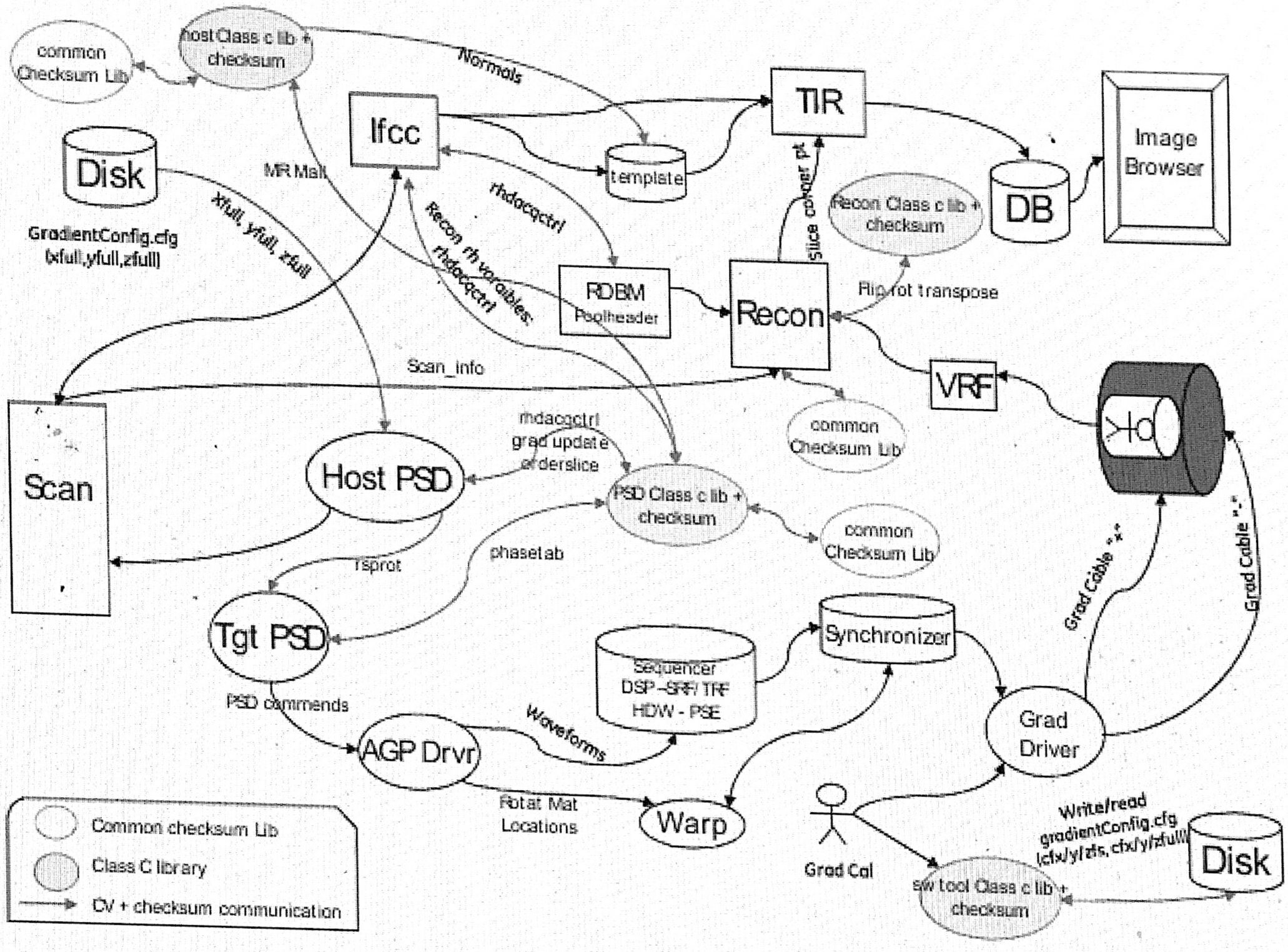


Figure 8-3: Image Orientation Data Flow

8.3. Patient Identification Protection

The hazard related to patient identification occurs if the incorrect patient information (patient ID, name, date of birth) is assigned to image data. The software components that could contribute to this hazard have been segregated from the remainder of the software.

The data passing between various subsystems is protected through the use of checksums. The subsystem which creates the patient information generates a checksum of the data. This checksum accompanies the data and is rechecked at image installation time to ensure data integrity. Additional integrity checks verifying the exam, series and image numbers are performed at the point where the pixel data is associated with the header data.

The hazard related to patient identification occurs if the incorrect patient information (patient ID, name, date of birth) is assigned to image data. The software components that could contribute to this hazard have been segregated from the remainder of the software.

The data passing between various subsystems is protected through the use of checksums. The subsystem which creates the patient information generates a checksum of the data. This checksum accompanies the data and is rechecked at image installation time to ensure data integrity. Additional integrity checks verifying the exam, series and image numbers are performed at the point where the pixel data is associated with the header data.

MR Recon clears all working memory between studies to ensure image data does not inadvertently cross study boundaries. Additionally, all accesses to working memory within MR Recon are validated to ensure the correct memory segments are being accessed.

The database subsystem ensures that image checksums exist in the DICOM image preamble. Images received via media or network transfers will have checksums added prior to database insertion. Images being installed from within the Scanner are checked against their respective checksum data.

The Viz and Applications software use the embedded checksums to verify that the images have not been corrupted. Additionally, they verify that the set of data received from a

*** Printed copies are uncontrolled unless otherwise identified ***
Before using this document, consult MyWorkshop for the latest revision.

database query match the expected patient. The Viz and Applications include the appropriate checksum information in all output derived images.

Invalid checksums associated with the image data consistency checks are considered unrecoverable errors. Processing is not allowed to continue and images exhibiting this failure will not be installed into the database. Images with invalid checksums will be saved into a "failed" directory for problem analysis.

Figure 8-4 shows the data paths and interactions associated with the patient information tied to the image data.

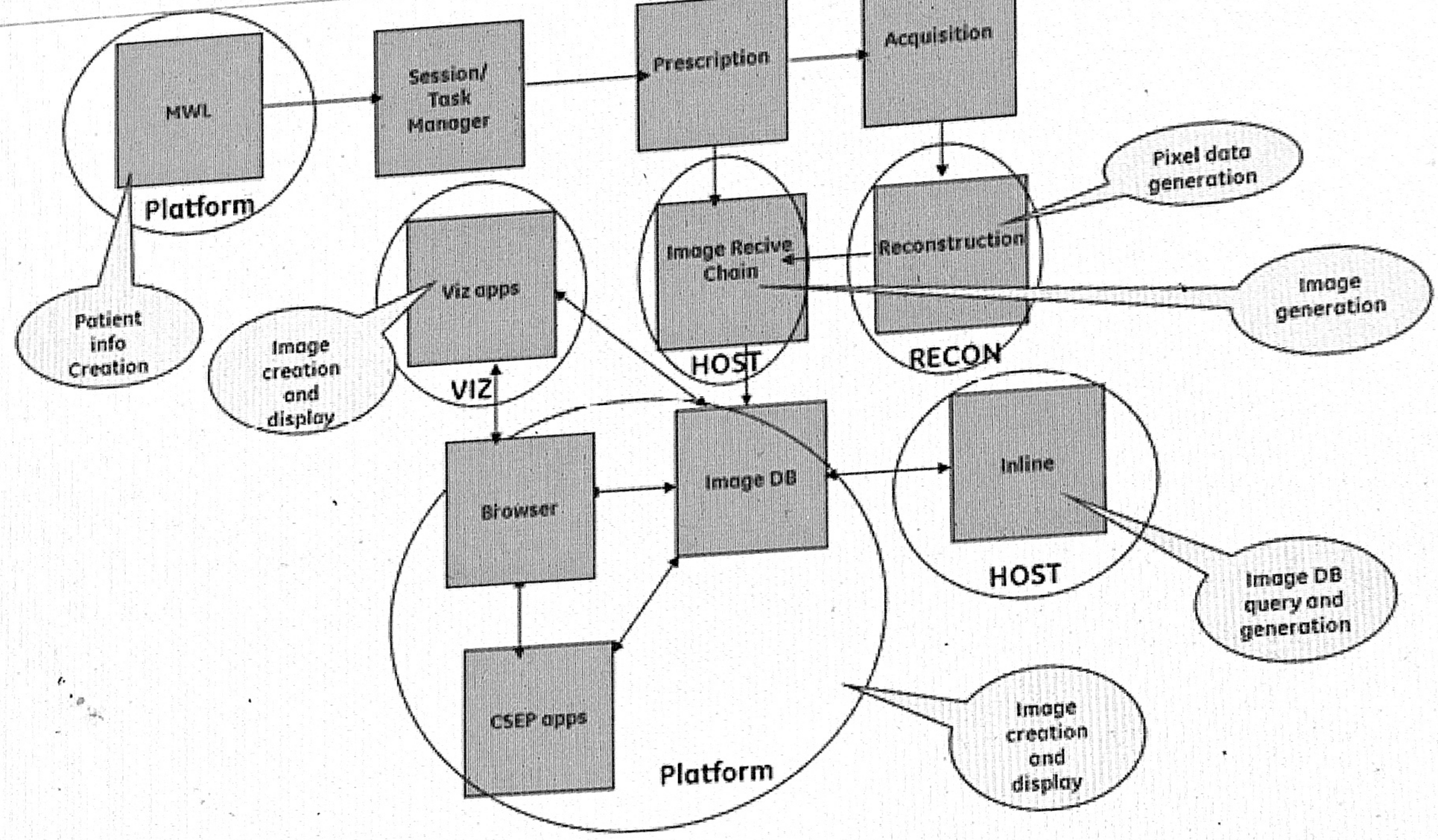


Figure 8-4: Patient ID Data Paths

9. GENERATION

This section describes the top-level look at the Software generation relative to tools used across subsystems and major SOUP components. The Software source files as well as the compilers and libraries are maintained under revision control. The build machines are capable of executing the cross-compilers to produce the object files for the hosting operating system on each of the specific subsystem computers. Refer to the component design documents for the locations and building mechanisms used for each component.

9.1. Tools

9.1.1. The tools used to develop the Software are often specialized for each software subsystem due, in part, to the compute environments in which

* Printed copies are uncontrolled unless otherwise identified *
Before using this document, consult MyWorkshop for the latest revision.

each subsystem operates. Subsystem-specific tools are described in the subsystem detailed design documents referenced in the Boost

All C++ software benefits from a variety of language enhancements not yet supported in the compiler versions used in product.

This design specification identifies Boost as the means to provide enhancements for more effective C++ code. Boost's capabilities range from general-purpose libraries like the smart pointer library, to CRC libraries, to operating system abstractions.

Structure section of this document (3.4.2).

Additional development tools are identified in the MR Waukesha CSV Tracker [7].

9.2. Directories and Files

The files associated with this software architecture are described in the detailed design documents within each subsystem.

9.3. SOUP

The Software uses many SOUP (Software Of Unknown Pedigree/Provenance) items. The majority of the SOUP used in the Software is specific to a single subsystem and is documented within the enclosing subsystem design. Several of the software items identified in the

Environment section of this document (3) are SOUP. Further details of the SOUP items are described in the Software Structure [15].

10. TRACEABILITY

T design is traced with requirements via references in this document and with subsystem designs via references in this document and references in the software subsystem design descriptions. Technical experts and/or functional architects will confirm the tracing by reviewing system-level requirements, this document, and all applicable software subsystem design documents.

End of Document